



इलेक्ट्रॉनिकी एवं
सूचना प्रौद्योगिकी मंत्रालय
MINISTRY OF
**ELECTRONICS AND
INFORMATION TECHNOLOGY**

सत्यमेव जयते

Empowering **Learners**. Strengthening **Skills**. Building the Digital **Future**.

NIELIT – SkillBank

Capacity Building & Skill Development Program

An Initiative of
Ministry of Electronics and Information Technology (MeitY)
to promote quality education, digital skills and employability
across India.



Industry Relevant
Training



Skill Enhancement
& Certification



Employability
& Growth



Innovation
& Excellence

Building Skills. Building Careers. Building **Bharat.**

Together for a Digital, Skilled and Empowered India

Introduction to Data Analytics in AI

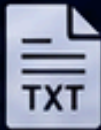
The Reality of Machine Learning

AI algorithms cannot process **raw databases or folders of images directly.**

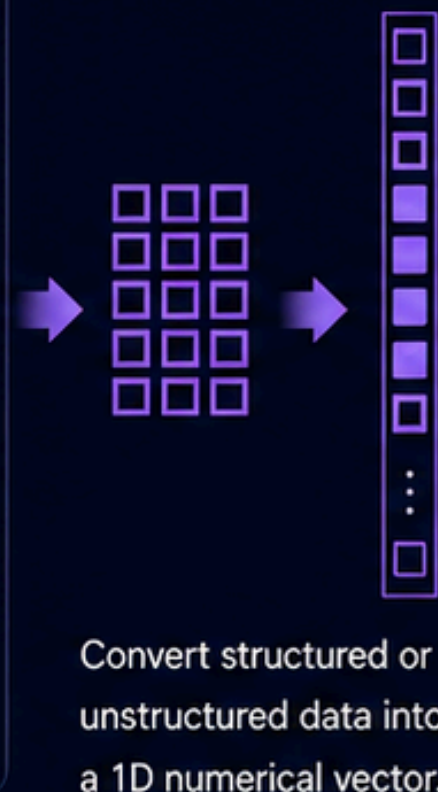
Every input—whether it is text morphology, video, or tabular data—must be mathematically flattened into **numerical arrays.**

Analytics is the process of **inspecting, cleaning, and transforming** this raw data before it reaches the neural network.

RAW DATA

-  Text (e.g., articles)
-  Video (e.g., surveillance)
-  Tabular Data (e.g., student scores)
-  Images (e.g., photos)
- ⋮

DATA FLATTENING

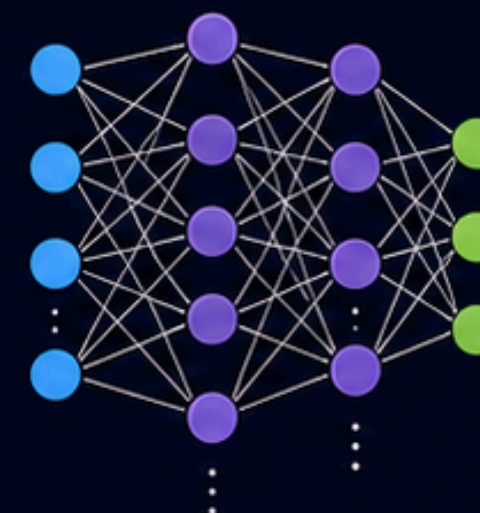


NUMERICAL ARRAY

$\begin{bmatrix} 0.23 \\ -1.05 \\ 3.14 \\ 0.00 \\ -0.87 \\ 2.71 \\ \vdots \end{bmatrix}$

A numerical representation that computers (and ML models) can understand.

NEURAL NETWORK



OUTCOMES / INSIGHTS

-  Classification
-  Regression
-  Insights & Decisions

THE ANALYTICS PIPELINE

1. INSPECT

Understand the data: types, shape, missing values, distributions, outliers, etc.

2. CLEAN

Handle missing values, remove duplicates, fix inconsistencies, correct errors.

3. TRANSFORM

Normalize, encode categorical data, scale features, aggregate, reshape, flatten, etc.

4. FEATURE ENGINEER

Create meaningful features that improve model performance.

5. VALIDATE

Ensure data quality and consistency before feeding into the model.

READY FOR MACHINE LEARNING



KEY TAKEAWAY

Great models don't start with algorithms—they start with great data.



Garbage In = Garbage Out



Clean data. Better models. Smarter AI.

GITHUB ORGANIZATION



<https://github.com/essentials-of-artificial-intelligence>

NumPy Architecture

Why Pure Python Fails at Scale



1 Python lists are arrays of pointers pointing to **scattered objects** in memory.



2 NumPy (Numerical Python) uses C-arrays containing **contiguous blocks** of identical data types.



3 This contiguous memory allows for **vectorized operations**, making math computations **orders of magnitude faster**.

PYTHON LIST (Scattered Memory)

A list is an array of pointers to Python objects



- Extra memory for pointers
- Objects scattered in memory
- Poor cache locality
- Slower operations

NUMPY ARRAY (Contiguous Memory)

A NumPy array is a contiguous block of memory



- Single contiguous memory block
- Better cache locality
- Less memory overhead
- Much faster operations

MEMORY VIEW



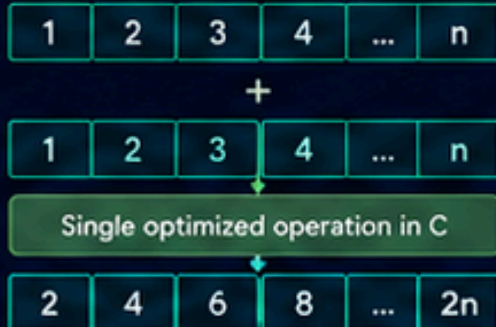
VECTORIZED OPERATIONS (Example: A + B)

Python List (Element-wise loop)



⌚ Slower (Python loop overhead)

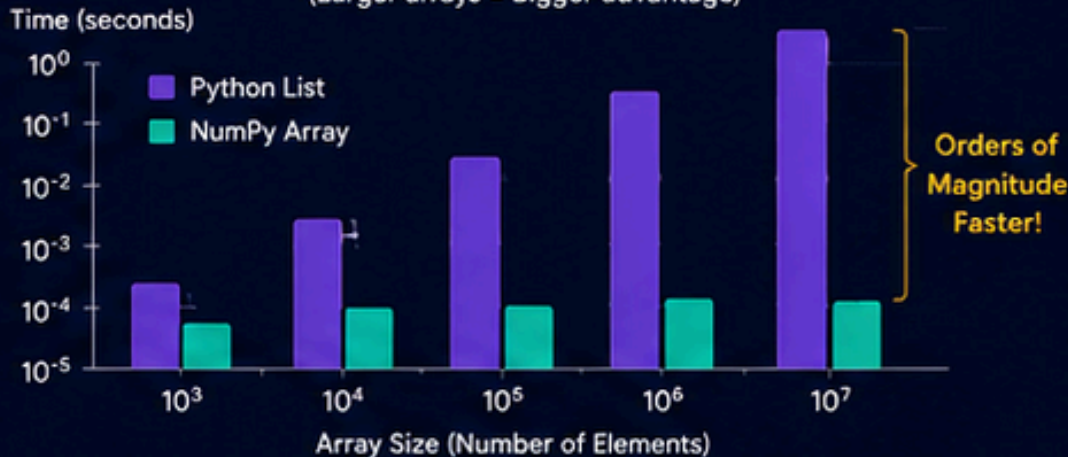
NumPy Array (Vectorized)



⚡ Much Faster (C under the hood)

PERFORMANCE COMPARISON

(Larger arrays = Bigger advantage)



KEY TAKEAWAY



NumPy's contiguous memory + vectorization
= Massive speed and memory efficiency.



The foundation for high-performance
data analytics and AI.



Less time waiting.
More time building intelligent solutions.

Vectors, Matrices, and Tensors

The Shape of Data



1 1D Array (Vector): A single sequence of values.

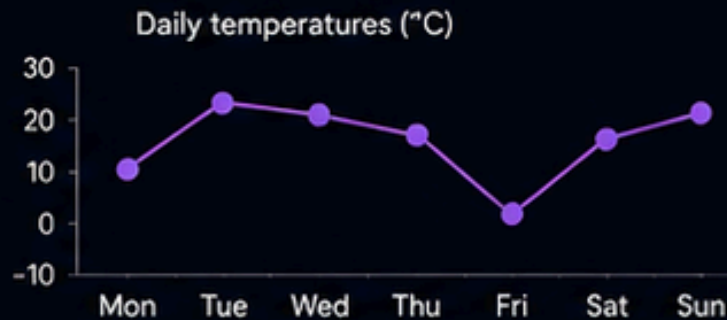
1D ARRAY (VECTOR)



NumPy Representation

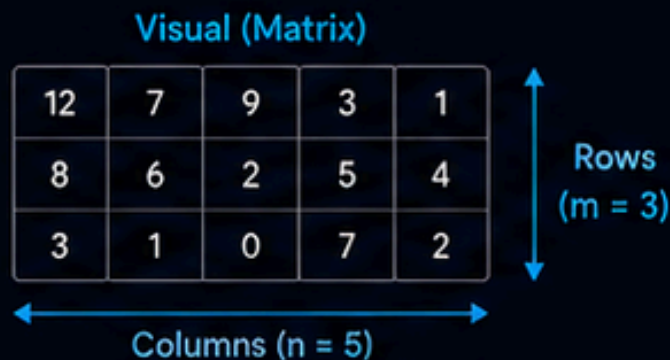
```
import numpy as np  
  
v = np.array([7.5, -2.3, 0.0,  
             4.1, 9.8, -1.2, 3.3])  
  
v.shape # (7,)
```

Real-World Example



2 2D Array (Matrix): Rows and columns (e.g., a grayscale image or a spreadsheet).

2D ARRAY (MATRIX)



NumPy Representation

```
import numpy as np  
  
M = np.array([[12, 7, 9, 3, 1],  
             [8, 6, 2, 5, 4],  
             [3, 1, 0, 7, 2]])  
  
M.shape # (3, 5)
```

Real-World Examples

Grayscale Image (pixel intensities 0-255)



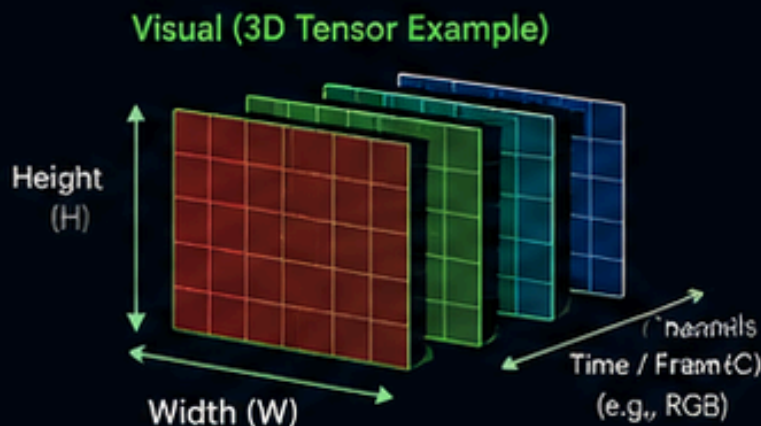
Spreadsheet / Table

Name	Math	Sci	Eng
Alice	85	90	78
Bob	92	88	81
Charlie	76	91	84



3 3D+ Array (Tensor): Deep dimensional arrays (e.g., RGB video frames or complex spatial coordinates).

3D+ ARRAY (TENSOR)



NumPy Representation

```
import numpy as np  
  
T = np.random.randint(0, 256,  
                      size=(120, 3, 64, 64),  
                      dtype=np.uint8)  
  
T.shape # (120, 3, 64, 64)
```

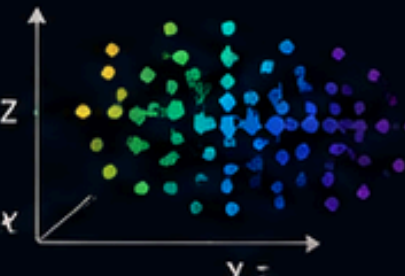
Frames (T) Channels (C) Height (H) Width (W)

Real-World Examples

RGB Video Frames



3D Point Cloud



Or any higher-dimensional data (e.g., medical scans, weather data cubes, robot joint trajectories, etc.)



Higher dimensions capture more information about the world.



NumPy gives us the power to store, manipulate, and compute efficiently.



From vectors to tensors—everything is just numbers in well-defined shapes.



Understand the shape. Unlock the power of your data.

Eliminating the for Loop



- 1 Vectorization:**
Applying operations to **entire arrays at once** rather than iterating through elements.



- 2 Broadcasting:**
NumPy dynamically **stretches smaller arrays** to match the shape of larger arrays during mathematical operations.

VECTORIZATION: OPERATE ON THE WHOLE ARRAY

THE SLOW WAY: FOR LOOP

```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])
c = np.empty_like(a)

for i in range(len(a)):
    c[i] = a[i] + b[i] # element-by-element
```

a	1	2	3	4	5
b	10	20	30	40	50
c	11	22	33	44	55

❗ Many Python-level operations (slow)

THE FAST WAY: VECTORIZED

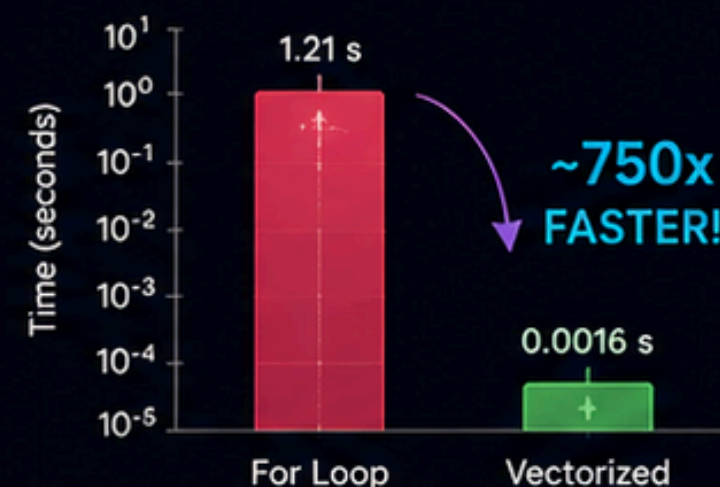
```
import numpy as np
a = np.array([1, 2, 3, 4, 5])
b = np.array([10, 20, 30, 40, 50])
c = a + b # one line, element-wise
```

a	1	2	3	4	5
b	10	20	30	40	50
=					
c	11	22	33	44	55

✅ One optimized operation in C (fast!)

PERFORMANCE COMPARISON

Add two arrays of size 10^7

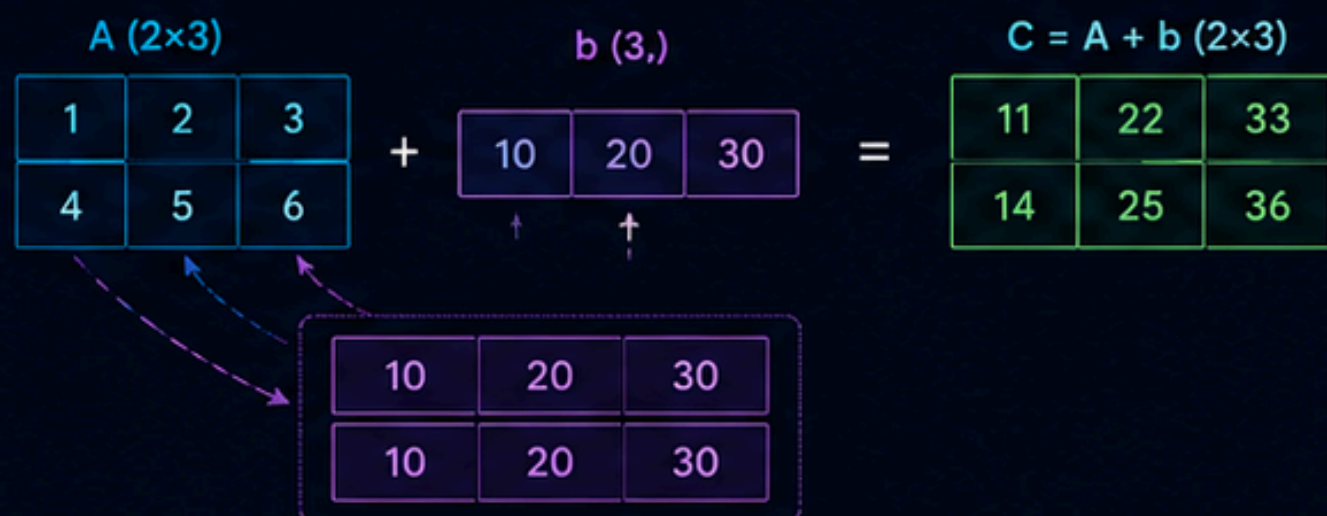


Vectorization moves the work + from Python to optimized C code.

BROADCASTING: NUMPY STRETCHES AUTOMATICALLY

EXAMPLE

```
A = np.array([[1, 2, 3],
              [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30]) # shape (3,)
C = A + b # Broadcasting!
```



BROADCASTING RULES (Simplified)

- 1 Compare shapes from the trailing (rightmost) dimensions.
- 2 Dimensions are compatible if they are equal or one of them is 1.
- 3 Dimensions of size 1 are stretched to match the other size.

MORE EXAMPLES

Scalar + Matrix

$$A (2 \times 3) + 5$$

1	2	3
4	5	6

+ 5

Column Vector + Matrix

$$A (2 \times 3) + \begin{bmatrix} 10 \\ 20 \end{bmatrix}$$

Matrix + Row Vector

$$A (2 \times 3) + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix}$$



Vectorization = Speed

Operate on whole arrays, not elements.



Fewer Python Operations

More work done in optimized C.



Broadcasting = Flexibility

Write concise code, NumPy handles the shape alignment.



Result: Faster Code, Cleaner Code, Better Performance.

High-Speed Filtering



1 Applying a condition to an array generates a “mask” of True/False values.



2 You can overlay this mask onto the original array to instantly extract only the data points that meet your criteria.



WHY IT'S FAST

No loops. No if-else.

Operations are performed in optimized C code under the hood.

Massive speed. Minimal code.

HOW BOOLEAN MASKING WORKS

1 Original Array

```
import numpy as np
data = np.array([12, 7, 25, 3, 18, 30, 9, 14, 21, 5])
```

data									
12	7	25	3	18	30	9	14	21	5
0	1	2	3	4	5	6	7	8	9

2 Create a Boolean Mask (Condition)

```
mask = data > 15
```

mask (data > 15)

F	F	T	F	T	T	F	F	T	F
0	1	2	3	4	5	6	7	8	9

T = True F = False

3 Apply the Mask to the Original Array

```
filtered = data[mask]
```

Overlay mask on data

12	7	25	3	18	30	9	14	21	5
0	1	2	3	4	5	6	7	8	9
×	×	✓	×	✓	✓	×	×	✓	✓

4 Filtered Result

```
filtered
```

25 18 30 21

Only values where mask is True are returned.

REAL-WORLD EXAMPLES



Filter Students with Score ≥ 70

```
scores = np.array([85, 42, 91, 65, 72, 33, 88])
```

```
scores  $\geq$  70
```

85	42	91	65	72	33	88
T	F	T	F	T	F	T

Result: [85, 91, 72, 88]



Filter Stock Prices Above \$100

```
prices = np.array([98.5, 101.2, 99.1, 120.0, 87.6])
```

```
prices > 100
```

98.5	101.2	99.1	120.0	87.6
F	T	F	T	F

Result: [101.2, 120.0]



Filter Temperatures Below 0°C

```
temps = np.array([5, -2, -8, 3, -1, 7])
```

```
temps < 0
```

5	-2	-8	3	-1	7
F	T	T	F	T	F

Result: [-2, -8, -1]

LOOP vs BOOLEAN MASKING

Using Python Loop (Slow)

```
result = []
for x in data:
    if x > 15:
        result.append(x)
```



Slow for large arrays
(O(n) in Python)

VS

Using Boolean Mask (Fast)

```
mask = data > 15
result = data[mask]
```



Fast & vectorized
(Optimized C code)



KEY TAKEAWAY

Boolean masking lets you filter huge datasets in a single line of code.



It's simple, readable, and incredibly fast.



Perfect for data cleaning, feature selection, and conditional analysis.



Filter smarter. Analyze faster.

Structured Data Analysis



1 Pandas is built directly on top of NumPy.

```
import pandas as pd
# Powered by NumPy under the hood
```



2 It introduces labels (column and row names) and SQL-like relational database operations.



3 It is the global industry standard for Exploratory Data Analysis (EDA) and feature engineering.

FROM NUMPY ARRAYS TO PANDAS DATAFRAME

NUMPY 2D ARRAY
No labels, just values

```
import numpy as np
arr = np.array([
    [101, "Alice", 23, 88.5],
    [102, "Bob", 20, 92.0],
    [103, "Charlie", 21, 79.5]
])
```

101	Alice	23	88.5
102	Bob	20	92.0
103	Charlie	21	79.5

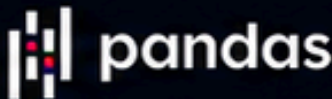
✗ Just a matrix of values

PANDAS DATAFRAME
Labeled, indexed, and powerful

```
import pandas as pd
df = pd.DataFrame(arr, columns=[
    "ID", "Name", "Age", "Score"
])
df.index = ["Row1", "Row2", "Row3"]
```

	ID	Name	Age	Score
Row1	101	Alice	23	88.5
Row2	102	Bob	20	92.0
Row3	103	Charlie	21	79.5

✓ Labeled, indexed, and easy to work with



Built on NumPy



High performance + Easy to use

SQL-LIKE RELATIONAL OPERATIONS



SELECT (Columns) `df[["Name", "Score"]]`



FILTER (Rows) `df[df["Age"] > 20]`



GROUP BY `df.groupby("Age").mean()`



JOIN (MERGE) `pd.merge(df1, df2, on="ID")`



SORT `df.sort_values("Score", ascending=False)`



REAL-WORLD ANALOGY

Think of Pandas DataFrame as an Excel spreadsheet + SQL database + high-performance engine.

WHERE PANDAS SHINES



Exploratory Data Analysis (EDA)
Inspect, summarize, visualize your data.



Data Cleaning
Handle missing values, duplicates, outliers, and inconsistencies.



Feature Engineering
Create, transform, and combine features for machine learning.



Data Integration
Merge, join, and reshape data from multiple sources easily.



Time Series Analysis
Powerful tools for time-indexed data.



Reporting & Export
Export to CSV, Excel, SQL DBs, or dashboards in one line.



KEY TAKEAWAY



NumPy gives you speed. Pandas gives you structure, labels, and intelligence.



From raw arrays to meaningful insights — that's Pandas.



Used by data scientists worldwide in every major data-driven industry.



Master Pandas. Master your data. Master your results.

Meeting the Dataset



1 Pandas reads CSVs, JSON, SQL, and Excel files natively with functions like `pd.read_csv()`.



2 `.head()` reveals the first 5 rows, while `.info()` displays data types and memory usage.



3 `.describe()` instantly generates summary statistics for every numerical column.

READ DATA FROM MULTIPLE SOURCES

CSV File



```
import pandas as pd
df = pd.read_csv('data.csv')
```

JSON File



```
df = pd.read_json('data.json')
```

Excel File



```
df = pd.read_excel('data.xlsx',
sheet_name='Sheet1')
```

SQL Database



```
import pandas as pd
import sqlalchemy as sa
engine = sa.create_engine('sqlite:///data.db')
df = pd.read_sql('SELECT * FROM table', engine)
```

SUPPORTED FORMATS



- ✓ Fast and optimized I/O
- ✓ Automatic parsing of headers and index
- ✓ Handles large datasets efficiently

.head() – FIRST 5 ROWS

df.head()

	ID	Name	Age	City	Salary
0	101	Alice	24	New York	60000
1	102	Bob	27	Los Angeles	55000
2	103	Charlie	22	Chicago	48000
3	104	David	29	Houston	62000
4	105	Eve	25	Phoenix	51000

Shows the first 5 rows (default). Great for a quick preview!

.info() – DATA TYPES & MEMORY USAGE

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   ID           5 non-null      int64
1   Name         5 non-null      object
2   Age          5 non-null      int64
3   City         5 non-null      object
4   Salary       5 non-null      int64

dtypes: int64(3), object(2)
memory usage: 332.0+ bytes
```

What it tells you:

- Column names
- Non-null counts
- Data types (float, object, etc.)
- Memory usage

.describe() – SUMMARY STATISTICS

df.describe()

	Age	Salary
count	5.000000	5.000000
mean	25.400000	55200.000000
std	2.607681	5460.819614
min	22.000000	48000.000000
25%	24.000000	51000.000000
50%	25.000000	55000.000000
75%	27.000000	60000.000000
max	29.000000	62000.000000

Instant insights for numerical columns:

- ✓ Count
- ✓ Mean
- ✓ Std Deviation
- ✓ Min / Max
- ✓ Quartiles (25%, 50%, 75%)

Perfect for quick EDA and understanding your data!

REAL-WORLD EXAMPLE: FROM FILE TO INSIGHT

sales.csv



pd.read_csv()



df.head()



df.info()



df.describe()



Understand Your Data



KEY TAKEAWAY



Load data from any source seamlessly.



Inspect structure, types, and sample data quickly.



Generate summary statistics in one line.



These three commands – **read**, **head**, **info**, **describe** – are your first tools to **explore** any dataset like a pro!



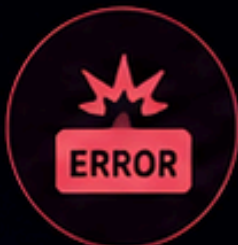
Data in. Insight out.

The Messy Reality of Data

Cleaning the Noise



1 Real-world data is plagued with missing values (NaN), duplicates, and formatting errors.



2 Models will crash if fed NaN values.



3 Solutions: Drop rows using .dropna().



4 Or impute values using .fillna().

FROM MESSY TO CLEAN

MESSY DATA (RAW)

	ID	Name	Age	Email	Salary
0	101	Alice	25	alice@example.com	50000
1	102	Bob	NaN	bob@example.com	NaN
2	103	Charlie	30	charlie@example.com	60000
3	103	Charlie	30	charlie@example.com	60000
4	104	David	twenty-eight	davidexample.com	55000
5	105	Eve	31	eve@example.com	NaN

Problems:

- Missing values (NaN)
- Duplicate rows (ID 103)
- Formatting errors (text instead of number, bad email)

CLEANED DATA

	ID	Name	Age	Email	Salary
	101	Alice	25	alice@example.com	50000
	102	Bob	NaN	bob@example.com	55000
	103	Charlie	30	charlie@example.com	60000
	105	Eve	31	eve@example.com	55000

What we fixed:

- Handled missing values
- Removed duplicates
- Corrected formatting issues



NaN values can break mathematical operations and cause models to crash or produce garbage!

Example: Model Crash

```
import numpy as np
arr = np.array([1, 2, np.nan, 4])
np.mean(arr) # Returns NaN → breaks training!
```

SOLUTIONS IN PANDAS

1. DROP ROWS WITH .dropna()

Remove rows that contain missing values.

```
df_clean = df.dropna()
# Drops any row with NaN
```

Result:

ID	Name	Age	Salary
101	Alice	25	50000
103	Charlie	30	60000
105	Eve	31	NaN



Simple and quick
Best when dataset is large



Loses data
May remove important rows

2. FILL / IMPUTE VALUES WITH .fillna()

Replace missing values with a value you define (mean, median, 0, etc.).

```
df_filled = df.fillna({
    'Age': df['Age'].median(),
    'Salary': df['Salary'].median()
})
```

Result:

ID	Name	Age	Salary
101	Alice	25.0	50000.0
102	Bob	27.5	55000.0
103	Charlie	30.0	60000.0
103	Charlie	30.0	60000.0
104	David	28.0	55000.0
105	Eve	31.0	55000.0



Keeps more data
Customizable strategies



Cons:
May introduce bias
Needs careful choice

COMMON IMPUTATION STRATEGIES



Mean

```
df['col'].fillna(df['col'].mean())
```

Good for symmetric numeric data.



Median

```
df['col'].fillna(df['col'].median())
```

Robust to outliers.



Mode

```
df['col'].fillna(df['col'].mode()[0])
```

Useful for categorical data.



Constant

```
df['col'].fillna(0)
```

Use with 0, 'Unknown', or any constant.



Forward Fill

```
df.fillna(method='ffill')
```

Fills with previous valid value.

DETECT ISSUES FIRST

```
df.isnull().sum()
```

→ Count missing values per column

```
df.duplicated().sum()
```

→ Count duplicate rows

```
df.dtypes
```

→ Check data types

```
df.value_counts()
```

→ Check value distribution



KEY TAKEAWAY



Dirty data in → garbage out.



Clean data is the foundation of great AI models.



Pandas gives you powerful tools to clean, fix, and prepare data for success.



Garbage in → Garbage out.
Clean in → Model win!

Data Transformation

Feature Engineering



1 The `.apply()` method executes custom functions across entire columns efficiently.



2 Mapping dictionaries to columns allows for fast **categorical encoding** (e.g., converting "Pass/Fail" into 1 and 0).



3 These transformations create **meaningful features** that boost model performance.

1. USING .apply() FOR CUSTOM TRANSFORMATIONS

Original DataFrame

	Name	Age	Income
0	Alice	25	50000
1	Bob	30	62000
2	Charlie	35	72000
3	David	28	54000
4	Eve	40	88000

Custom Function
(e.g., categorize income)

```
def income_bucket(x):  
    if x < 50000:  
        return 'Low'  
    elif x < 80000:  
        return 'Medium'  
    else:  
        return 'High'
```

Apply Function to Column

```
df['Income_Bucket'] =  
df['Income'].apply(income_bucket)
```

	Name	Age	Income	Income_Bucket
0	Alice	25	50000	Medium
1	Bob	30	62000	Medium
2	Charlie	35	72000	Medium
3	David	28	54000	Medium
4	Eve	40	88000	High

⚡ `.apply()` is vectorized and optimized in C under the hood — much faster than Python loops!

MORE .apply() EXAMPLES

Extract Year from Date

```
df['Year'] = df['Join_Date'].apply(  
    lambda x: x.year)
```

Create Age Group

```
df['Age_Group'] = df['Age'].apply(  
    lambda x: '18-25' if x <= 25  
    else '26-35' if x <= 35 else '36+')
```

Clean Text (Strip + Title Case)

```
df['Name_Clean'] = df['Name'].apply(  
    lambda x: x.strip().title())
```

2. MAPPING DICTIONARIES FOR FAST CATEGORICAL ENCODING

Original Data

Student	Result	Attendance	Grade
Alice	Pass	Present	A
Bob	Fail	Absent	C
Charlie	Pass	Present	B
David	Fail	Absent	D
Eve	Pass	Present	A

Mapping Dictionaries

Result Map
{
 'Pass': 1,
 'Fail': 0
}

Attendance Map
{
 'Present': 1,
 'Absent': 0
}

Apply Mapping

```
df['Result_Encoded'] =  
df['Result'].map(result_map)  
  
df['Attendance_Encoded'] =  
df['Attendance'].map(att_map)
```

Transformed Data

Student	Result_Encoded	Attendance_Encoded	Grade
Alice	1	1	A
Bob	0	0	C
Charlie	1	1	B
David	0	0	D
Eve	1	1	A

✅ `.map()` is faster than `.apply()` for simple lookups!

WHY USE MAPPING?

- ✔ Fast and memory-efficient
 - ✔ Easy to read and maintain
 - ✔ Consistent encodings
 - ✔ Essential for ML models
-

REAL-WORLD EXAMPLES

E-commerce
Map categories (Electronics, Fashion) to numeric codes

Healthcare
Map diagnosis (Yes/No) to 1 and 0

Finance
Map risk levels (Low/Med/High) to 0/1/2

NLP
Clean text, extract length, remove punctuation

KEY TAKEAWAY

`.apply()` lets you run custom logic across entire columns.

Mapping dictionaries provide blazing-fast categorical encoding.

Smart transformations turn raw data into powerful model-ready features.

The Split-Applpy-Combine Strategy



1 `.groupby()` splits the data into categories, applies a statistical function (like mean or sum), and combines the results.



2 Essential for extracting tiered statistics, such as finding the average assignment score broken down by specific engineering tracks.

1. SPLIT
Split data into groups based on one or more keys.

Student	Track	Assignment	Score
Alice	AI/ML	A1	85
Bob	AI/ML	A1	90
Charlie	Data	A1	78
David	Data	A1	88
Eve	DevOps	A1	92
Frank	DevOps	A1	75
Grace	AI/ML	A2	95
Heidi	Data	A2	82
Ivan	DevOps	A2	87

2. APPLY
Apply a function to each group (independently).

AI/ML (3 rows)

Alice	85
Bob	90
Grace	95

Data (3 rows)

Charlie	78
David	88
Heidi	82

DevOps (3 rows)

Eve	92
Frank	75
Ivan	87

3. COMBINE
Combine all the results into a new DataFrame.

Apply Function (e.g., mean)

`mean(85, 90, 95)`
= 90.00

`mean(78, 88, 82)`
= 82.67

`mean(92, 75, 87)`
= 84.67

Combined Result

Track	Avg_Score (mean)
AI/ML	90.00
Data	82.67
DevOps	84.67

Code

```
import pandas as pd
df.groupby('Track')['Score'].mean()
# Returns the combined result
```

OTHER AGGREGATIONS

Track	count	mean	sum	min	max
AI/ML	3	90.00	270	85	95
Data	3	82.67	248	78	88
DevOps	3	84.67	254	75	92

Multiple Aggregations

```
df.groupby('Track')['Score'].agg(['count', 'mean', 'sum', 'min', 'max'])
```

Powerful summaries in a single line!

GROUP BY MULTIPLE KEYS

Track	Assignment	mean(Score)
AI/ML	A1	87.50
AI/ML	A2	95.00
Data	A1	83.00
Data	A2	82.00
DevOps	A1	83.50
DevOps	A2	87.00

```
df.groupby(['Track', 'assignment'])['Score'].mean().reset_index(name='Avg_Score')
```

- ✓ Extract meaningful insights at any level of granularity
- ✓ Perfect for dashboards, reports, and ML features
- ✓ Scales to millions of rows efficiently



KEY TAKEAWAY



SPLIT
Break data into meaningful groups.



APPLY
Compute statistics on each group.



COMBINE
Merge results into a clean, interpretable table.



`.groupby()` is the backbone of real-world data analysis.